

Renderer w oparciu o grafy zadań z automatycznym zarządzaniem zasobami i synchronizacją GPU

Spis Treści

Spis Treści	1
Opis projektu	2
Etapy projektu	3
ETAP 1 - Fundamenty	3
Funkcjonalności	3
Zależności	3
Podział pracy	3
ETAP 2 - Synchronizacja	4
Funkcjonalności:	4
Zależności	4
Podział pracy	4
ETAP 3 - Lifetime & Transient Resources	4
Funkcjonalności	5
Zależności	5
Podział pracy	5
ETAP 4 - Aliasing + Allocator	5
Funkcjonalności	5
Zależności	5
Podział pracy	5
ETAP 5 - Pass Culling + optymalizacje	6
Funkcjonalności	6
Zależności	6
Podział pracy	6
ETAP 6 - Demo / Integracja	6
Funkcjonalności	6
Zależności	7
Podział pracy	7
Bibliografia	7

Opis projektu

Celem pracy inżynierskiej jest zaprojektowanie oraz implementacja silnika renderującego 3D wykorzystującego koncepcję grafu zadań (Render Graph).

We wczesnych systemach renderujących stosowano wysokopoziomowe API graficzne, które upraszczały zarządzanie renderingiem, jednak jednocześnie ograniczały możliwości optymalizacji. Współczesne aplikacje, w szczególności systemy czasu rzeczywistego, takie jak gry komputerowe, wymagają maksymalnej wydajności oraz pełnej kontroli nad wykorzystaniem zasobów sprzętowych. W związku z tym coraz częściej wykorzystuje się niskopoziomowe API (np. Vulkan), które umożliwiają bardziej precyzyjne zarządzanie GPU, kosztem większej złożoności implementacyjnej.

Metody renderowania grafiki 3D można podzielić ze względu na liczbę przejść przetwarzania danych. Wyróżnia się przede wszystkim renderowanie bezpośrednie (forward rendering) oraz renderowanie odroczone (deferred rendering). W podejściu forward rendering obiekty są przetwarzane w jednym przejściu, co upraszcza implementację oraz zmniejsza zużycie pamięci, jednak ogranicza możliwości obsługi złożonego oświetlenia. Z kolei deferred rendering dzieli proces renderowania na wiele etapów (tzw. passów), takich jak obliczenia geometrii, przetwarzanie pikseli czy operacje obliczeniowe. Podejście to umożliwia efektywne wykorzystanie wielu źródeł światła w czasie rzeczywistym, jednak wiąże się z większym zapotrzebowaniem na pamięć oraz koniecznością zarządzania zależnościami pomiędzy poszczególnymi etapami.

Renderowanie odroczone, ze względu na swoją wieloetapowość, wymaga zaawansowanego zarządzania zasobami wykorzystywanymi podczas generowania pojedynczej klatki obrazu. W kontekście renderowania zasobami nazywamy obiekty przechowujące dane w pamięci GPU, takie jak tekstury oraz bufor. Kluczowym problemem związanym z ich wykorzystaniem jest synchronizacja dostępu, czyli zapewnienie poprawnej kolejności operacji odczytu i zapisu.

Celem projektu jest rozwiązanie tego problemu poprzez zastosowanie grafu zadań, w którym przejścia renderujące definiowane są w sposób deklaratywny. Każde przejście określa wykorzystywane zasoby oraz operacje na nich wykonywane. Na tej podstawie system automatycznie wyznacza kolejność wykonywania operacji oraz generuje odpowiednie bariery synchronizacyjne, zapewniając poprawną współpracę CPU i GPU.

Dodatkowym elementem projektu jest implementacja mechanizmów zarządzania pamięcią, w tym techniki memory aliasing. Polega ona na współdzieleniu tej samej przestrzeni pamięci GPU przez różne zasoby, o ile ich czas życia nie pokrywa się w trakcie wykonywania grafu. Pozwala to znacząco zmniejszyć zużycie pamięci, co jest istotne szczególnie w przypadku złożonych scen renderowanych w czasie rzeczywistym.

Projekt zakłada również stworzenie modularnej architektury systemu, umożliwiającej łatwą wymianę backendu graficznego. Podstawową wspieraną technologią będzie Vulkan, jednak architektura systemu pozwoli na implementację dodatkowych backendów, takich jak DirectX czy Metal.

System zostanie zaimplementowany w języku C++, z wykorzystaniem podejścia Data-Oriented Design w kluczowych elementach odpowiedzialnych za reprezentację przejść renderujących, zasobów oraz zależności pomiędzy nimi. Celem takiego podejścia jest poprawa lokalności danych oraz efektywniejsze wykorzystanie pamięci podręcznej procesora.

W celu testowania oraz prezentacji działania systemu przygotowane zostanie proste środowisko testowe (sandbox), umożliwiające ładowanie modeli i innych zasobów wykorzystywanych w procesie renderowania. Środowisko to będzie pozwalało na rekonfigurację grafu renderowania, w szczególności na włączanie i wyłączanie wybranych przejść oraz analizę ich wpływu na poprawność i wydajność działania systemu.

Etapy projektu

ETAP 1 - Fundamenty

W tym etapie celem jest stworzenie podstawowej struktury Render Graphu oraz mechanizmu definiowania i wykonywania passów.

Zostanie zaprojektowane API dla **Render Pass**, umożliwiające dodawanie passów do grafu oraz deklarowanie użycia zasobów (read/write). Na tej podstawie powstanie graf zależności (DAG), który zostanie posortowany topologicznie, aby zapewnić poprawną kolejność wykonania.

Na końcu etapu graf będzie możliwy do wykonania w najprostszej formie (bez optymalizacji i synchronizacji).

Funkcjonalności

- Definicja RenderPass (API)
- Dodawanie passów do graphu
- Rejestracja resource (Texture/Buffer)
- Read / Write usage tracking
- Budowa dependency graph (DAG)
- Topological sort
- Proste wykonanie passów (bez optymalizacji)

Zależności

- To jest etap bazowy → blokuje wszystkie kolejne etapy

Podział pracy

- Osoba A
 - Projekt RenderPass API
 - Dodawanie passów
- Osoba B (Graph / algorytmy)

- DAG + topological sort
- Osoba C (Resources)
 - Resource registry + read/write tracking

ETAP 2 - Synchronizacja

W tym etapie celem jest zapewnienie poprawnej synchronizacji operacji GPU poprzez analizę konfliktów dostępu do zasobów.

System wykryje hazardy (read/write), a następnie wygeneruje odpowiednie bariery synchronizacji. Całość zostanie zintegrowana z backendem (np. Vulkan) poprzez komponent **BarrierEmitter**.

Funkcjonalności:

- Analiza read/write hazardów
- Generowanie barierów
- Integracja z backendem (Vulkan)
- BarrierEmitter

Zależności

- Wymaga ETAPU 1 (read/write tracking + kolejność passów)

Podział pracy

- Osoba A (analiza)
 - Hazard detection
- Osoba B (backend Vulkan)
 - Implementacja barierów (np. vkCmdPipelineBarrier)
- Osoba C (abstrakcja)
 - BarrierEmitter (API niezależne od backendu)

ETAP 3 - Lifetime & Transient Resources

W tym etapie celem jest automatyczne zarządzanie cyklem życia zasobów oraz optymalizacja pamięci.

System będzie śledził moment pierwszego i ostatniego użycia zasobu, co pozwoli na automatyczne tworzenie i zwalnianie zasobów. Wprowadzone zostaną również zasoby tymczasowe (transient).

Funkcjonalności

- Śledzenie first/last usage resource
- Automatyczne tworzenie/usuwanie zasobów
- Transient resources (per frame)
- Resource registry

Zależności

- Wymaga ETAPU 1
- Może być rozwijany równolegle z ETAPEM 2 (nie blokują się bezpośrednio)

Podział pracy

- Osoba A (lifetime analysis)
 - first/last usage
- Osoba B (resource system)
 - registry + lifecycle
- Osoba C (integration)
 - podpięcie pod graph execution

ETAP 4 - Aliasing + Allocator

W tym etapie celem jest redukcja zużycia pamięci GPU poprzez ponowne wykorzystanie zasobów.

Na podstawie analizy lifetime'ów zasobów możliwe będzie ich aliasowanie (współdzielenie pamięci). Zostanie również zaimplementowany prosty allocator oraz system pooling'u.

Funkcjonalności

- Analiza nakładania lifetime
- Reużycie pamięci GPU
- Pooling zasobów
- Prosty allocator (np. linear + reuse)

Zależności

- Wymaga ETAPU 3

Podział pracy

- Osoba A (analysis)
 - wykrywanie overlapów lifetime
- Osoba B (allocator)
 - implementacja allocatora

- Osoba C (integration)
 - podpięcie aliasingu pod resource system

ETAP 5 - Pass Culling + optymalizacje

W tym etapie celem jest eliminacja zbędnych operacji w grafie renderowania.

System będzie usuwał passy i zasoby, które nie mają wpływu na końcowy wynik, co pozwoli na poprawę wydajności.

Funkcjonalności

- Usuwanie nieużywanych passów
- Analiza final outputs
- Dead resource elimination

Zależności

- Wymaga ETAPU 1
- Korzysta z ETAPU 3, ale może działać częściowo niezależnie

Podział pracy

- Osoba A (graph traversal)
 - analiza od outputów
- Osoba B (pass culling)
 - usuwanie passów
- Osoba C (resource cleanup)
 - usuwanie martwych zasobów

ETAP 6 - Demo / Integracja

W tym etapie celem jest integracja systemu oraz jego demonstracja w praktyce.

Zostanie przygotowany prosty renderer wykorzystujący Render Graph oraz narzędzia do debugowania grafu.

Funkcjonalności

- Prostý renderer:
 - G-buffer
 - lighting pass
 - postprocess
- Debug view graphu

Zależności

- Wymaga wszystkich poprzednich etapów

Podział pracy

- Osoba A (rendering)
 - G-buffer + lighting
- Osoba B (postprocess)
 - efekty końcowe
- Osoba C (debug tools)
 - wizualizacja graphu

Bibliografia

1. https://www.youtube.com/watch?v=uQQzDRFKm_4
2. <https://www.youtube.com/watch?v=FBYq64QKjFo>
3. <https://www.youtube.com/watch?v=pr8HalaZfpk>
4. https://asawicki.info/news_1738_states_and_barriers_of_aliasing_render_targets
5. <https://github.com/skaarj1989/FrameGraph>
6. <https://www.gdcvault.com/play/1024612/FrameGraph-Extensible-Rendering-Architecture-in>
7. <https://www.slideshare.net/slideshow/framegraph-extensible-rendering-architecture-in-frostbite/72795495>
8. <https://www.youtube.com/watch?v=qjWkNZ0SXfo>
9. <https://doc.babylonjs.com/features/featuresDeepDive/frameGraph/frameGraphBasicConcepts/>
10. <https://logins.github.io/graphics/2021/05/31/RenderGraphs.html>
11. https://en.wikipedia.org/wiki/Data-oriented_design
12. [https://en.wikipedia.org/wiki/Aliasing_\(computing\)](https://en.wikipedia.org/wiki/Aliasing_(computing))
13. <https://theknightsofu.com/render-graph-introduction/>
14. <https://github.com/DragonJoker/RenderGraph>